Übungsblatt 2 — Bäume (2)

Aufgabe 1: AVL-Bäume – Einfügen und Löschen

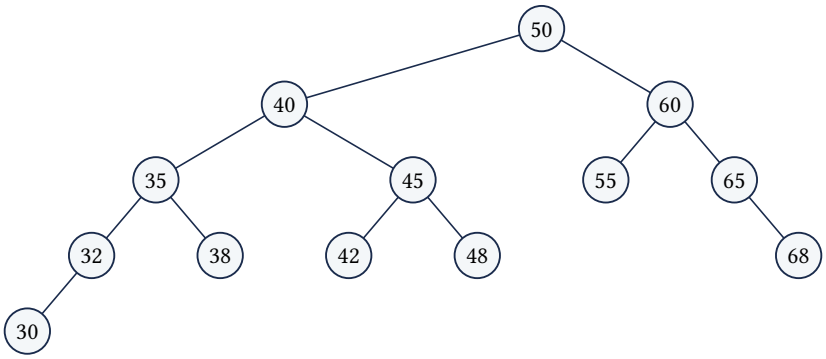
Gegeben sei ein leerer AVL-Baum.

1. Konstruieren Sie den AVL-Baum durch Einfügen der Schlüssel in der Reihenfolge: `30, 40, 50, 20, 25`
2. Löschen Sie den Knoten `50` und anschließend den Knoten `25`.
3. **Optional**: Gegeben sei erneut ein leerer AVL-Baum. Fügen Sie die Schlüssel `60, 50, 40, 70, 65` ein.

> **Hinweis:** Betrachten Sie das Einfügen und Löschen jedes Schlüssels als eigene Operation und prüfen Sie danach die AVL-Bedingung. Benennen Sie jede Ausgleichsmaßnahme (z. B. L(40)).

Aufgabe 2: AVL-Bäume – Löschen mit Kaskade

Gegeben sei der folgende AVL-Baum.

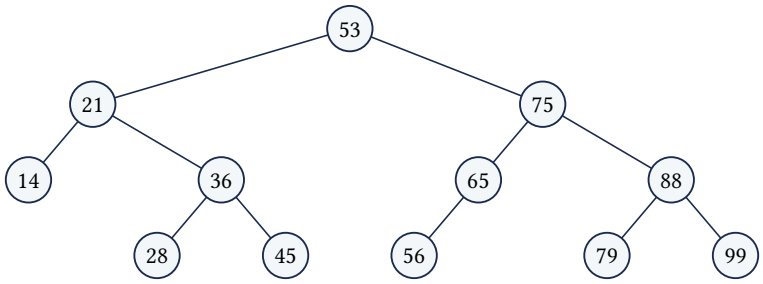


1. Ergänzen Sie die Balancefaktoren der Knoten.
2. Löschen Sie den Knoten 55.

> **Hinweis:** Löschen kann zu kaskadierenden Ausgleichsmaßnahmen führen: Nach einer Rotation kann sich die Höhe des Teilbaums verringert haben, dann muss der Weg bis zur Wurzel weiter geprüft werden.

Aufgabe 3: AVL-Bäume - Löschen (weitere Fälle)

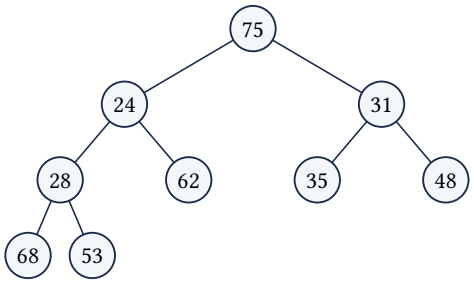
Gegeben sei der folgende AVL-Baum.



1. Ergänzen Sie die Balancefaktoren der Knoten.
2. Löschen Sie nacheinander die Knoten `14`, `45` und `28`. Geben Sie jeweils an, ob und welche Ausgleichsmaßnahme nötig ist.

Aufgabe 4: Heaps korrigieren

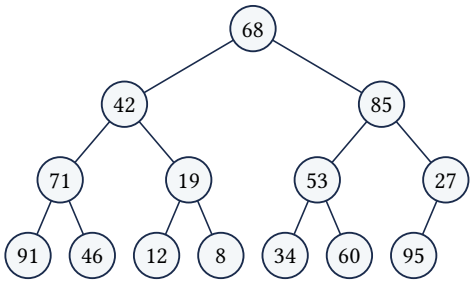
Gegeben seien der folgenden fast vollständigen (Min-)Heap, in dem genau ein Knoten falsch ist.



Ermitteln Sie den falschen Knoten und korrigieren Sie den Heap durch Anwendung eines bekannten Algorithmus.

Aufgabe 5: Heaps – BuildHeap

Gegeben sei der folgende binäre Baum.



1. Erklären Sie kurz in eigenen Worten den Algorithmus `BuildHeap`.
2. Erzeugen Sie aus dem gegebenen Baum unter Anwendung bekannter Algorithmen einen Heap.

> **Hinweis:** Beim Angeben von Zwischenergebnissen reicht die Kurzschreibweise.

Aufgabe 6: Treaps – Einfügen und Löschen

Gegeben sei ein leerer Treap.

1. Fügen Sie die folgenden Schlüssel-Prioritäts-Paare in der gegebenen Reihenfolge ein: `(10,5), (20,2), (5,7), (15,1), (3,9), (25,4), (12,3)`
2. Löschen Sie nacheinander die Elemente `(25,4)`, `(10,5)` und `(15,1)` aus dem bestehenden Treap.

> **Hinweis:** Notation L((k,p)) bzw. R((k,p)) ist die Links- bzw. Rechtsrotation. k nach Suchbaum, p nach Min-Heap.

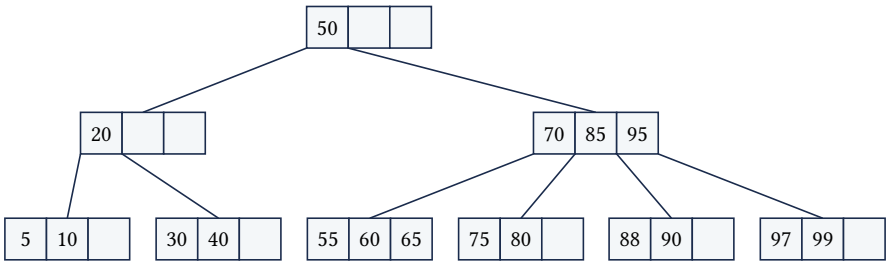
Aufgabe 7: Treaps – Eindeutigkeit und Rotationen

Gegeben sei die Menge von Schlüssel-Prioritäts-Paaren:
`S = {(2,7), (4,3), (6,9), (8,5), (10,1), (12,8), (14,4), (16,6), (18,2)}`.

1. Konstruieren Sie den Treap für S direkt, keine Zwischenschritte nötig.
2. Fügen Sie in den Treap aus Teil 1 das Element `(11,0)` ein und illustrieren Sie alle Zwischenschritte.
3. **Optional**: Löschen Sie aus dem Treap aus Teil 1 das Element `(4,3)`.
4. Begründen Sie, warum der Treap bei gegebenen Schlüssel-Prioritäts-Paaren (bei paarweise verschiedenen Schlüsseln und Prioritäten) unabhängig von der Einfügereihenfolge immer derselbe ist.

Aufgabe 8: B-Bäume – Ordnung und Eigenschaften

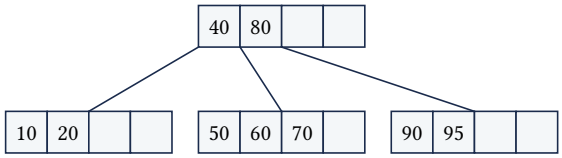
Gegeben sei der folgende B-Baum.



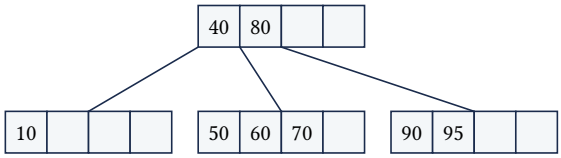
1. Welche Ordnung d hat dieser B-Baum? Wie viele Elemente muss eine Seite mindestens und höchstens enthalten (Wurzel und übrige Seiten)? Wie viele Nachfolger kann eine innere Seite haben?
2. Verfolgen Sie die Suche (`BTreeSearch`) nach den Elementen `80` und `62`. Geben Sie jeweils die besuchten Seiten und das Ergebnis an. Was fällt bei der erfolglosen Suche auf?

Betrachten Sie nun die folgenden Bäume.

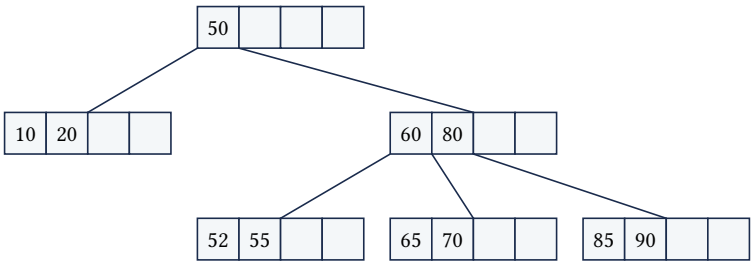
a)



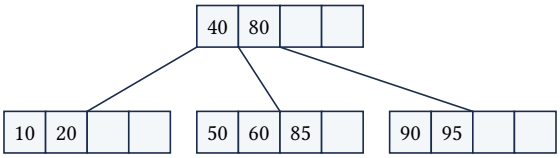
b)



c)



d)



3. Entscheiden Sie für jeden der Bäume a) bis d), ob er ein B-Baum der Ordnung 5 ist. Falls nicht, nennen Sie die verletzte Eigenschaft.

Aufgabe 9: B-Bäume – Einfügen & Löschen

Gegeben sei ein leerer B-Baum der Ordnung 3.

1. Fügen Sie die folgenden Schlüssel in der gegebenen Reihenfolge ein: `10, 20, 30, 40, 50, 60, 70, 15, 65`
2. Löschen Sie nacheinander die Schlüssel `15, 20, 50` aus dem bestehenden B-Baum.